



BY

J. STEVE MANUEL

HEXAWARE - GET TRAINEE

DATA ENGINEERING

APACHE AIRFLOW

Apache Airflow is an open-source platform used to programmatically author, schedule, and monitor workflows, primarily for data engineering pipelines. It uses Python to define workflows as Directed Acyclic Graphs (DAGs) of tasks, enabling robust scheduling, dependency management, and monitoring via a web-based user interface

- These data pipelines are used to deliver datasets that are easily used by business intelligence applications or machine learning models where a huge amount of data is required.
- It is one of the most robust platforms for data engineers.
- Batch-oriented workflows are developed, scheduled, and monitored efficiently. Apache Airflow is a workflow engine that easily schedules and runs complex data pipelines

Key Advantages of Apache Airflow:

- **Pipeline as Code (Python):** Workflows are defined in Python, enabling dynamic pipeline generation, easy version control, testing, and reuse.
- **Highly Scalable & Flexible:** Airflow can handle small to massive workflows using distributed task execution. It supports horizontal scaling to meet changing data needs.
- **Extensive Integrations & Connectors:** It includes built-in operators and hooks for various technologies, including Apache Spark, Kubernetes, SQL databases, and major cloud providers (AWS S3, Google Cloud Storage).
- **Rich User Interface (UI):** Provides a comprehensive web-based UI for real-time monitoring, managing workflows, triggering tasks, analyzing task failures, and viewing logs.
- **Robust Task Scheduling & Dependency Management:** Airflow allows definition of complex dependencies between tasks, including branching, retries on failure, and sensor-based execution (e.g., waiting for a file to arrive).
- **Open Source & Strong Community:** As a mature, Apache-licensed project, it benefits from a large, active community providing regular updates, documentation, and support.

Core Architectural Components

Apache Airflow is built on a modular microservices architecture designed to orchestrate complex data workflows. Its core design centers on **Directed Acyclic Graphs (DAGs)**, which represent workflows as code, ensuring tasks are executed in a specific order with clear dependency management.

Airflow's architecture consists of several primary components that work together to manage, schedule, and execute tasks:

- **Scheduler:** The "brain" of Airflow. It monitors all DAGs and their tasks, triggering instances when dependencies are met and schedules are reached.
- **Web Server:** The "face" of Airflow. It provides a user interface (UI) to inspect, trigger, and debug DAGs, as well as view logs and manage the environment.
- **Metadata Database:** The "memory" of Airflow. Usually a relational database like [PostgreSQL](#) or [MySQL](#), it stores the state of tasks, DAG runs, variables, and configurations.
- **Executor:** A property of the scheduler that determines *how* and *where* tasks get done. It can run tasks locally or distribute them across a cluster.
- **Workers:** The "hands" of Airflow. These are separate processes or nodes that execute the actual logic defined in the tasks.
- **Triggerer (Optional):** Executes deferred tasks in an asynchronous event loop, allowing for efficient waiting on external events

Execution Flow

1. **Parsing:** The **DAG Processor** (often part of the scheduler) scans the DAGs folder to parse Python files and update the metadata database.
2. **Scheduling:** The **Scheduler** identifies tasks ready to run and moves them to a "Scheduled" state in the database.
3. **Queuing:** Ready tasks are handed to the **Executor**, which places them in a queue (like [Redis](#) or [RabbitMQ](#) in distributed setups).
4. **Execution:** **Workers** pull tasks from the queue, execute them, and report their final status (Success/Failure) back to the metadata database.
5. **Monitoring:** The **Web Server** reads from the metadata database and log files to display real-time status to the user.

What is a DAG (Directed Acyclic Graph)?

A DAG is a conceptual structure used to model workflows, data pipelines, or processing steps. It ensures that tasks are executed in a specific order without infinite loops.

- **Directed:** Edges have a specific direction (A ->B), meaning task A must finish before B begins.
- **Acyclic:** There are no loops or cycles (A -> B -> C -> A is not allowed).
- **Graph:** Composed of nodes (tasks) and edges (dependencies).

In Data Engineering (e.g., Apache Airflow): A DAG defines the order of task execution (e.g., "Run Data Ingestion" -> "Run Transformation" ->"Run Report Generation").

In Distributed Computing (e.g., Apache Spark): A DAG represents the logical sequence of operations (transformations like map or filter) applied to data before an action is triggered.

Key Benefit: DAGs make it easy to parallelize independent tasks and handle retries if a specific step fails.

What are Executors?

Executors are distributed worker processes responsible for running the tasks assigned by the driver application (in Spark) or the scheduler (in Airflow). They are the "muscle" that executes the code.

- **Launch on Workers:** Executors run on worker nodes within a cluster.
- **Execute Tasks:** They receive tasks from the scheduler, run them, and return results.
- **Manage Resources:** They provide memory (storage) and CPU cores to execute tasks.
- **Pluggable:** In systems like Apache Airflow, executors can be swapped (e.g., LocalExecutor for testing, CeleryExecutor or KubernetesExecutor for production).